

SHOUT-Trainer: Closed-loop Trainer for Silent Data Corruption Hunting and Observation Using Transformers

S. Maryam Ghasemi*, Shanmukha Mangadahalli Siddaramu*, Mehdi B. Tahoori
 Department of Computer Science, Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany
 E-mail: {seyedehmaryam.ghasemi, shanmukha.siddaramu, mehdi.tahoori}@kit.edu

Abstract—Silent data corruptions (SDCs) pose a significant reliability challenge in modern computing systems as they silently propagate incorrect results without triggering architectural exceptions or system failures. Fault injection (FI) is the de facto approach for quantifying SDC behavior, but large-scale fault injection campaigns are computationally prohibitive and difficult to apply continuously during software development. In this work, we propose a learning-based framework that predicts the probability of SDC propagation directly from source code using large language models trained on FI-annotated programs. To improve prediction accuracy, we introduce a closed-loop training strategy in which weakly predicted SDC program sites are identified and used to guide targeted FI campaigns on the corresponding programs. The resulting FI data is incorporated into the training dataset, creating a closed-loop retraining cycle. Experimental evaluation shows that the closed-loop predictor reduces the mean absolute error by 43.3% on the poorly predicted SDC program sites of the open-loop model, confirming that targeted feedback improves prediction accuracy in the most challenging regions of the program space.

Index Terms—Silent Data Corruption, Silent Data Error, Hardware Fault Model, Large Language Model, Transformer

I. INTRODUCTION

Silent Data Corruptions (SDCs) have become a major reliability threat in large-scale computing systems, where processors silently produce incorrect outputs without triggering crashes, error correction codes, or architectural exceptions [1], [2]. Recent fleet studies report manufacturing test escapes approaching ~5K Defective Parts per Million (DPM), with nearly 1K DPM manifesting as SDC-causing devices and producing hundreds of thousands of incorrect outputs per billion chip-hours in production deployments [3]. These failures propagate through operating systems and applications as corrupted data, offering no hardware indication of malfunction and requiring substantial engineering effort to diagnose [4]–[7]. A central challenge is that many SDCs are triggered only under specific instruction sequences, operand patterns, or compiler optimizations that activate marginal microarchitectural behavior [8]. While system-level monitors can detect corrupted outputs, they provide little visibility into the software regions that triggered or amplified the corruption. Consequently, developers must analyze large code bases to localize vulnerable instruction patterns, a process that is slow, expensive, and often fleet-wide [2].

Fault Injection (FI) remains the standard methodology for quantifying software reliability. By injecting faults into program execution and classifying outcomes (masked, crash, SDC), FI provides precise resilience metrics and enables estimation of the empirical SDC probability at individual program sites [9].

However, FI campaigns are computationally intensive and scale poorly, since each new program version, input set, or compiler configuration requires repeated injections. This makes FI impractical as an interactive tool during software development and limits its applicability for rapidly evolving software systems.

Prior work has studied SDC across several abstraction levels, from Register Transfer Level (RTL), microarchitectural, and architectural simulations to instruction-level, compiler-level, and application-level analyses [2], [6], [10]–[21]. These studies show that SDCs arise from complex interactions between hardware behavior and software execution, motivating cross-layer modeling and FI methodologies. Although program-level fault injection may not fully capture the propagation of transient hardware faults [22], software-level FI remains a practical and scalable approach for dependability evaluation and algorithmic vulnerability analysis [23]–[25]. Recently, Large Language Models (LLMs) have been applied to hardware design, verification, and Electronic Design Automation (EDA) tasks [26]–[31], but existing work mostly focuses on design assistance, debugging, or instruction-level analysis rather than predicting SDC vulnerability directly from software source code. Unlike prior studies, this work treats source code as the prediction target and uses prediction errors themselves to guide new FI campaigns, turning FI into targeted feedback for model improvement in identifying program structures that may increase SDC susceptibility.

In this work, we propose SHOUT, a framework for Silent Data Corruption Hunting and Observation Using Transformers, that predicts the probability of SDC propagation directly from source code, trained on a diverse corpus of FI-annotated programs. An automated fault injection campaign is conducted to produce per-operation SDC probability labels, which are used to fine-tune CodeBERT, a transformer encoder, to regress SDC probability from source code context alone. To address systematic prediction errors, we introduce a closed-loop retraining strategy: program sites where the predictor's absolute error exceeds a threshold are passed to a Re-trainer LLM, which generates semantically targeted code variants designed to amplify or suppress SDC behavior at those operations. These variants undergo a second targeted FI campaign, and the resulting labeled data is merged into the training set for retraining, closing the loop. Experimental results show that this feedback mechanism achieves a 43.3% reduction in MAE on the weakly predicted program sites of the open-loop model, confirming that targeted data augmentation effectively improves prediction accuracy in the most challenging regions of the operation space.

*These authors contributed equally to this work

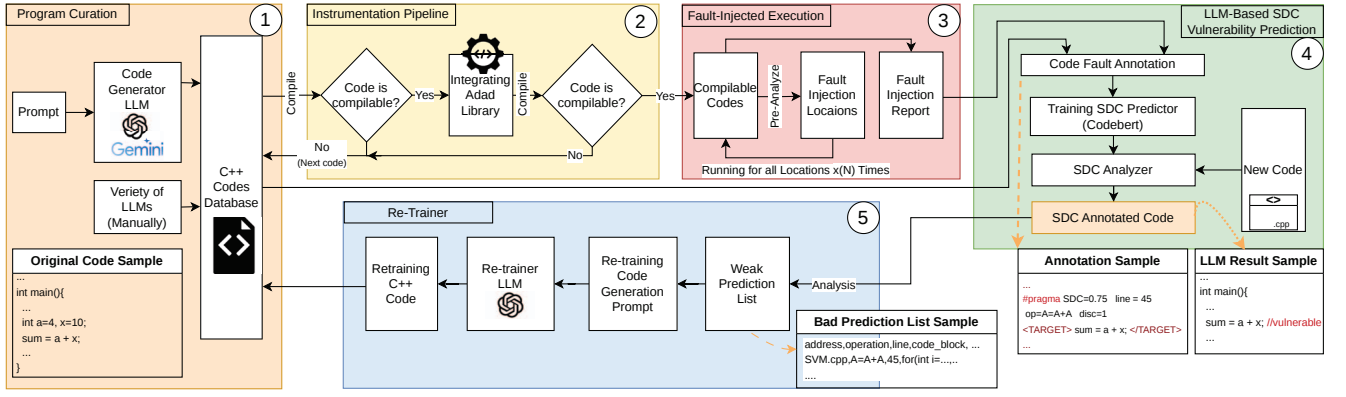


Fig. 1: An overview of the designed system, including 1) code instrumentation, 2) fault injection, 3) dataset generation, 4) training the LLM-based SDC predictor, and 5) the re-trainer.

II. PROPOSED APPROACH

The SHOUT-Trainer methodology follows a closed-loop process for SDC prediction and improvement. It begins with code generation and collection. Then, through a fully automatic process, and faults are then injected to generate labeled data for SDC analysis. A transformer encoder is trained to predict SDC vulnerability. Finally, the prediction results are analyzed to generate a new set of training programs, annotated with FI data, creating a feedback loop that improves SDC prediction accuracy. Fig. 1 shows this flow that is discussed in the following subsections.

A. Program Curation

A diverse corpus of programs is constructed to ensure broad coverage of control-flow and data-flow patterns. The dataset includes a large number of programs generated by state-of-the-art LLMs, including ChatGPT, Claude, and Gemini, covering tasks such as hashing, searching, numerical computation, sorting, machine learning, cryptography, and graph processing and publicly available implementations from Rosetta Code [32].

B. Instrumentation Pipeline

To provide feedback at the source code level, we perform fault injection at the same level. For the programming language, C/C++ was chosen, as they dominate systems software, infrastructure, core runtimes such as CPython in Python, and operating systems. However, the same approach can also be applied to other programming languages, including Python.

The fault injection framework used in this work is called *adad*, a custom developed library. It allows the user to assign a fault probability to each individual operation in the program. Faults are injected at the operation level rather than at the syntactic operator level. The framework operates as follows:

- **Fault injection method** — The fault model specifies how errors are introduced within the library. While the framework is compatible with various fault models, this study employs the widely used single bit-flip model [33], [34]. Assume that a dynamic instruction or loop iteration is executed n times and that a fault occurs independently with probability p during each execution. The resulting probability of silent data corruption (P_{SDC}) based on the

probability of SDC in case the fault happens on iteration k ($P_{SDC}(it : k)$), can be written as

$$P_{SDC}(p) = \sum_{k=1}^n p(1-p)^{n-1} P_{SDC}(it : k) + \mathcal{O}(p^2) \quad (1)$$

For realistic soft-error rates, $p \ll 1$. Therefore higher-order terms corresponding to multiple faults can be neglected, leading to the first-order approximation

$$P_{SDC}(p) \approx p \sum_{k=1}^n P_{SDC}(it : k) \quad (2)$$

Consequently, estimating the SDC probability only requires evaluating the effect of single faults at individual dynamic instances.

Also, the calculation done on Spearman rank correlation between normalized iteration index and SDC occurrence for each program suggests that uniformly random sampling of injection iterations is a valid and unbiased strategy for fault injection experiments. Due to space constraints, a detailed analysis is not included in this manuscript. In this work, faults are therefore injected into a single randomly selected loop iteration.

- **Library integration**— The example in Fig. 2 demonstrates that modifying the original source code is minimal, requiring only the inclusion of a header file and updating the relevant variable declarations.

C. Fault-Injected Execution

By selecting a fault site and specifying the fault model, the framework executes a run with the corresponding modeled fault. For each fault injection site (l), it records the number of executions and the observed behavior, including assigning a unique ID, the operation, and its line number, enabling precise per-operation characterization of software-level vulnerability.

Fault-labeled data are generated through a Monte Carlo fault injection campaign, following these steps:

- Execute a fault-free run to establish the reference output and the number of times each fault injection site was called during the run.
- Perform a large Monte Carlo analysis for each distinct injection site and on different randomly selected iterations.

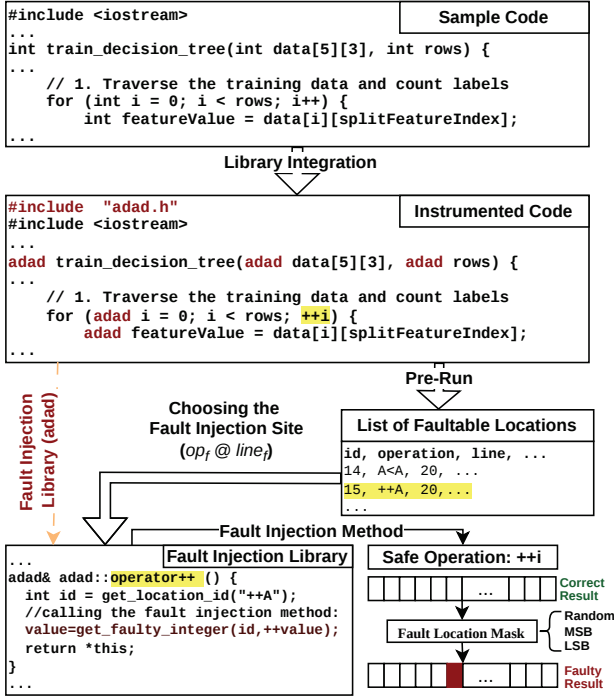


Fig. 2: Steps of the fault injection campaign, including code instrumentation, overloading the example increment (++) operator in the adad library, and performing fault injection.

- For every run, record the injection site and whether the outcome is correct (masked), an SDC, or a DUE (crash or timeout), and store all metadata in a structured table.

D. LLM-Based SDC Probability Prediction

Predicting whether a program operation will produce an SDC is challenging, as the impact of a fault depends not only on the local instruction but also on the surrounding program context. These interactions are highly non-linear, making accurate prediction difficult for conventional static analysis techniques. Therefore, we fine-tune a transformer encoder CodeBERT to adapt its latent representation from general program semantics to fault-aware semantics [35]. Each fault injection site (l) is associated with an empirical SDC probability $P_{\text{SDC}}(l) \in [0, 1]$, and the model is trained to regress this probability from source code context alone.

1) *Training Phase*: Each training entry pairs a code block (x_l) with its continuous label $P_{\text{SDC}}(l)$. The target operation at line l is wrapped with `<TARGET>` and `</TARGET>` tags, and for sequences exceeding the 512-token limit, the window is centered on the `<TARGET>` token to preserve the most relevant context. The tokenized input is passed through CodeBERT (f_θ), a 12-layer transformer encoder pre-trained on code, and the [CLS] token representation is used as the aggregate embedding $h_l = f_\theta(x_l)[\text{CLS}]$. A regression head (g_ϕ) then maps this 768-dimensional embedding to a predicted SDC probability:

$$\hat{P}_\theta(l) = \sigma(g_\phi(h_l)) \in [0, 1]$$

where g_ϕ consists of $\text{Linear}(768 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.4) \rightarrow \text{Linear}(256 \rightarrow 1)$, and the sigmoid output con-

strains predictions to the valid probability range. The model is trained to minimize a weighted mean squared error loss.

2) *Deployment Phase*: Given unseen source code, per-site contexts x_l are extracted by centering the context window on each `<TARGET>`-marked operation. The tokenized input is passed through the fine-tuned encoder and regression head to produce a continuous SDC probability estimate $\hat{P}_\theta(l) \in [0, 1]$ per injection site.

E. Re-trainer Feedback Loop

Following the analysis of the SDC predictor described in Sec. II-D, program sites with the highest prediction errors are collected into a weak prediction list, along with their corresponding code blocks and predicted and actual SDC rates. This list is passed to a Re-trainer LLM, which generates N new code variants per site, each preserving the structural logic of the original code block around the `<TARGET>`-marked operation. The behavioral mode of the generator is set according to the direction of the error: *amplify* mode targets sites where $\hat{P}_{\text{SDC}}$ underestimates the actual SDC rate, prompting the LLM to produce code more likely to propagate faults, while *mask* mode targets overestimated sites, producing code more likely to mask or crash. The generated programs are subjected to a targeted FI campaign, and the resulting labeled data is merged with $\mathcal{D}_{\text{OL}}$ to retrain the SDC predictor, closing the loop.

III. SIMULATION SETUP AND RESULTS

A. Simulation Setup

To generate labeled supervision for the learning model, we conduct a large scale fault injection campaign on a corpus of approximately 1.5 million lines of C++ code spanning more than 13,000 programs. Each injection site l is associated with an empirical SDC probability $p_{\text{SDC}}(l) \in [0, 1]$, estimated from $N = 100$ repeated fault injections. Faults are injected into four operator classes commonly associated with SDC propagation: arithmetic, comparison, boolean, and assignment operations. This study focuses on 32 bit integer operations, which dominate control flow and arithmetic behavior in typical C/C++ programs. The methodology is not restricted to this type and can be extended to floating point types and additional operator classes with minimal modification. The initial open loop campaign ($\mathcal{M}_{\text{OL}}$) produces the dataset $\mathcal{D}_{\text{OL}}$ containing 110K labeled fault sites. Analysis of its predictions reveals weakly predicted SDC program sites. These program sites are flagged by the system and used to guide a targeted second injection campaign on the corresponding programs. Model performance is measured using mean absolute error (MAE) root mean squared error (RMSE), and the coefficient of determination (R^2).

B. Open-Loop Prediction Performance

The open-loop predictor $\mathcal{M}_{\text{OL}}$ is trained exclusively on $\mathcal{D}_{\text{OL}}$ and evaluated on its held-out validation set, achieving MAE = 0.171, RMSE = 0.258, $R^2 = 0.521$, and Pearson correlation $r = 0.742$. To contextualize these results, we compare against two simple baselines. A global mean predictor that always outputs the average SDC probability yields $R^2 = 0$ and MAE = 0.349. A per-operation mean predictor, which predicts the

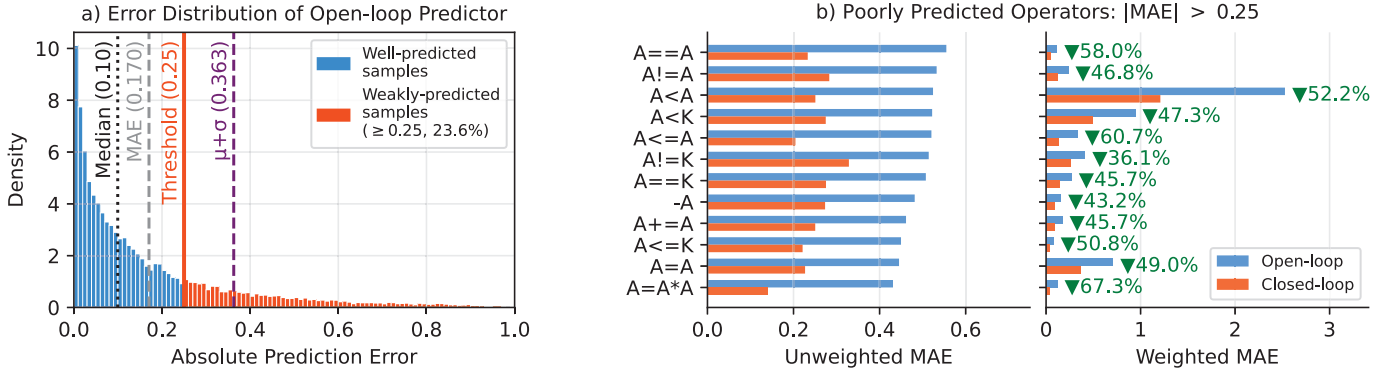


Fig. 3: (a) Absolute error distribution of the open-loop predictor $\mathcal{M}_{OL}$. (b) Per-operation MAE comparison for the 12 of 16 operator types with the largest prediction errors between $\mathcal{M}_{OL}$ and $\mathcal{M}_{CL}$. Unweighted MAE is defined per operator class k as $\frac{1}{n_k} \sum_{i \in k} |\hat{y}_i - y_i|$, reflecting prediction accuracy within each class. Weighted MAE scales by relative frequency, $\frac{n_k}{N} \cdot \text{MAE}_k$, quantifying each operator's contribution to overall model error, where n_k is the number of samples for operator k and N is the total number of validation samples.

average SDC rate for each operator type, achieves $R^2 = 0.085$ and $\text{MAE} = 0.327$. The fact that operation type alone explains only 8.5% of the variance indicates that the performance of $\mathcal{M}_{OL}$ ($R^2 = 0.521$) arises primarily from learning program-level structural patterns rather than relying on simple operation-type heuristics.

The theoretical noise floor arising from estimating SDC probability using $N = 100$ fault-injection runs per site is approximately 0.026. Despite this intrinsic uncertainty in the labels and the complex program interactions governing SDC propagation, $\mathcal{M}_{OL}$ achieves an MAE of 0.171, indicating that the model captures meaningful code-level signals for predicting SDC vulnerability. Figure 3 shows the absolute error distribution of $\mathcal{M}_{OL}$. Most predictions cluster near zero error, while a long right tail corresponds to weakly predicted program sites.

C. Performance Improvement through Closed-Loop Training

To construct the feedback loop, validation sites from $\mathcal{M}_{OL}$ with absolute prediction error

$$|\hat{P}_{\mathcal{M}_{OL}}(l) - P_{\text{SDC}}(l)| \geq 0.25$$

are identified as weakly predicted program sites. The threshold lies between the median error and the first standard deviation boundary of the error distribution, capturing approximately 24% of the worst predicted program sites.

This criterion identifies approximately 5K weakly predicted SDC program sites spanning 3.6K programs. A second fault injection campaign targeting these programs produces the feedback dataset $\mathcal{D}_{FB}$, containing 21K additional labeled program sites. The feedback dataset exhibits a significantly different distribution, with mean $P_{\text{SDC}} = 0.65$ and only 15% zero-SDC sites, compared with 31% in $\mathcal{D}_{OL}$. This confirms that the feedback mechanism successfully targets a more challenging subset of the operation space dominated by vulnerable operations.

Retraining the model using the combined dataset ($\mathcal{D}_{CL} = \mathcal{D}_{OL} \cup \mathcal{D}_{FB}$) produces the closed-loop predictor $\mathcal{M}_{CL}$. The retrained model achieves $\text{MAE} = 0.146$, $\text{RMSE} = 0.254$, and $R^2 = 0.61$, corresponding to a 6% reduction in MAE and a 17% improvement in R^2 relative to $\mathcal{M}_{OL}$. Notably,

the improvement on the poorly predicted program sites of the open-loop predictor is substantial, as $\mathcal{M}_{CL}$ reduces MAE from 0.460 to 0.261, corresponding to a 43.3% reduction. This confirms that the feedback dataset $\mathcal{D}_{FB}$ effectively targets the most challenging prediction regions.

To analyze which operators benefit the most from this feedback augmentation, MAE is computed per operation class for operators with at least 50 validation samples. We focus on the 12 of the 16 samples for which $\mathcal{M}_{OL}$ exhibits the largest prediction errors. Across these operators, $\mathcal{M}_{CL}$ consistently reduces MAE, with improvements ranging from 36.1% ($A \neq K$) to 67.3% ($A = AA$). The weighted MAE analysis further shows that $A < A$ contributes the largest aggregate prediction burden due to its high frequency among difficult samples, yet $\mathcal{M}_{CL}$ still reduces its weighted error by 52.2%. The uniform improvement across operator types indicates that the closed-loop training strategy improves generalization rather than overfitting to the specific samples used to construct $\mathcal{D}_{FB}$.

IV. CONCLUSIONS

SDCs pose a growing reliability challenge in modern computing systems, yet identifying software regions that trigger or amplify these faults remains difficult at scale. In this work, we explore a learning-based approach that predicts SDC probability directly from source code using large language models trained on FI-annotated programs. To improve prediction accuracy, we introduce a closed-loop training strategy in which weakly predicted SDC program sites are flagged to guide targeted fault injection campaigns. New program variants are automatically generated for operations with high prediction error, and FI is performed on these programs to produce additional labeled training data. The resulting FI data is incorporated into the training dataset, creating a continuous cycle of prediction, targeted data generation, and retraining. Experimental results show that this feedback mechanism significantly improves prediction quality, achieving a 43.3% reduction in MAE on the poorly predicted SDC program sites of the open loop predictor. This confirms that the feedback dataset $\mathcal{D}_{FB}$ effectively targets the most challenging prediction regions and enables more accurate SDC vulnerability estimation.

REFERENCES

- [1] S. Mitra, S. S. Banerjee, M. Dixon, M. Fuller, R. Govindaraju, P. Hochschild, E. X. Liu, B. Parthasarathy, and P. Ranganathan, "Silent data corruption by 10× test escapes threatens reliable computing," *IEEE Design Test*, vol. 42, no. 6, pp. 40–53, 2025.
- [2] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, "Silent data corruptions at scale," *arXiv preprint arXiv:2102.11245*, 2021.
- [3] S. Mitra, S. Banerjee, M. Dixon, M. Fuller, R. Govindaraju, P. Hochschild, E. X. Liu, B. Parthasarathy, and P. Ranganathan, "Silent data corruption by 10× test escapes threatens reliable computing," *IEEE Design & Test*, 2025.
- [4] N. Saxena, S. Hukerikar, and S. Hari, "Silent data corruption—optimal mitigation strategies for datacenter computing," *IEEE Micro*, pp. 1–10, 2025.
- [5] H. D. Dixit, L. Boyle, G. Vunnam, S. Pendharkar, M. Beadon, and S. Sankar, "Detecting silent data corruptions in the wild," *arXiv preprint arXiv:2203.08989*, 2022.
- [6] S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production cpu population," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 216–230.
- [7] A. Singh, S. Chakravarty, G. Papadimitriou, and D. Gizopoulos, "Silent data errors: Sources, detection, and modeling," in *2023 IEEE 41st VLSI Test Symposium (VTS)*, 2023, pp. 1–12.
- [8] E. J. Marinissen, H. D. Dixit, S. Blanton, A. Kuo, W. Li, S. Mitra, C. Nigh, R. Purdy, B. Kaczer, D. Sangani et al., "Silent data corruption: Test or reliability problem?" in *2024 IEEE European Test Symposium (ETS)*. IEEE, 2024, pp. 1–7.
- [9] B. Wen, J. Gu, D. Shen, Q. Zhou, F. Zhuang, Y. Liu, H. Song, and X. Huang, "Efficient Instruction Vulnerability Prediction With Heterogeneous SDC Propagation Knowledge Graph," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 1173–1190, Jan. 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/11179941/>
- [10] D. Mueller-Gritschneider, U. Sharif, and U. Schlichtmann, "Performance and accuracy in soft-error resilience evaluation using the multi-level processor simulator etiss-ml," in *ICCAD*. IEEE, 2018, pp. 1–8.
- [11] A. Rohani, "Modelling and mitigation of soft-errors in cmos processors," 2014.
- [12] G. Papadimitriou and D. Gizopoulos, "Silent data corruptions: Microarchitectural perspectives," *IEEE Transactions on Computers*, vol. 72, no. 11, pp. 3072–3085, 2023.
- [13] A. Singh, S. Chakravarty, G. Papadimitriou, and D. Gizopoulos, "Silent data errors: Sources, detection, and modeling," in *2023 IEEE 41st VLSI Test Symposium (VTS)*. IEEE, 2023, pp. 1–12.
- [14] G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Silent data corruptions: The stealthy saboteurs of digital integrity," in *IOLTS*. IEEE, 2023, pp. 1–7.
- [15] D. Gizopoulos, G. Papadimitriou, O. Chatzopoulos, N. Karystinos, H. D. Dixit, and S. Sankar, "Silent data corruptions in computing systems: Early predictions and large-scale measurements," in *2024 IEEE European Test Symposium (ETS)*. IEEE, 2024, pp. 1–10.
- [16] J. Calhoun, L. Olson, and M. Snir, "Flipit: An llvm based fault injector for hpc," in *Euro-Par 2014*. Springer, 2014, pp. 547–558.
- [17] S. K. S. Hari, S. V. Adve, and H. Naeimi, "Low-cost program-level detectors for reducing silent data corruptions," in *IEEE/IFIP*, 2012, pp. 1–12.
- [18] M. Didehban, H. So, P. Gali, A. Shrivastava, and K. Lee, "Generic soft error data and control flow error detection by instruction duplication," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 1, pp. 78–92, 2023.
- [19] P. H. Hochschild, P. Turner, J. C. Mogul, R. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat, "Cores that don't count," in *Proceedings of the Workshop on Hot Topics in Operating Systems*, ser. HotOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 9–16. [Online]. Available: <https://doi.org/10.1145/3458336.3465297>
- [20] Z. Li, H. Menon, D. Maljovec, Y. Livnat, S. Liu, K. Mohror, P.-T. Bremer, and V. Pascucci, "Spotsdc: Revealing the silent data corruption propagation in high-performance computing systems," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 10, pp. 3938–3952, 2020.
- [21] N. Yang, S. Jiang, Y. Wang, and Q. Miao, "Sd2ci: A structure-driven approach for detecting silent data corruption instructions," *Journal of Electronic Testing*, 2025.
- [22] G. Papadimitriou and D. Gizopoulos, "Demystifying the System Vulnerability Stack: Transient Fault Effects Across the Layers," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. Valencia, Spain: IEEE, Jun. 2021, pp. 902–915. [Online]. Available: <https://ieeexplore.ieee.org/document/9499847/>
- [23] R. Natella, D. Cotroneo, and H. S. Madeira, "Assessing dependability with software fault injection: A survey," *ACM Computing Surveys (CSUR)*, vol. 48, no. 3, pp. 1–55, 2016.
- [24] H. A. haj Ahmad and Y. Sedaghat, "An automated framework for selectively tolerating sdc errors based on rigorous instruction-level vulnerability assessment," *Future Generation Computer Systems*, vol. 157, pp. 392–407, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24001365>
- [25] M. Eslami, B. Ghavami, M. Raji, and A. Mahani, "A survey on fault injection methods of digital integrated circuits," *Integration*, vol. 71, pp. 154–163, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016792601930402X>
- [26] R. Ma, Y. Yang, Z. Liu, J. Zhang, M. Li, J. Huang, and G. Luo, "VerilogReader: LLM-Aided Hardware Test Generation," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10691801/>
- [27] C. Hong, S. Bhatia, A. Haan, S. K. Dong, D. Nikiforov, A. Cheung, and Y. S. Shao, "LLM-Aided Compilation for Tensor Accelerators," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–14. [Online]. Available: <https://ieeexplore.ieee.org/document/10691748/>
- [28] S. Qiu, B. Tan, and H. Pearce, "LLM-aided explanations of EDA synthesis errors," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10691721/>
- [29] L. J. Wan, H. Ye, J. Wang, M. Jha, and D. Chen, "An Iteratively-refined Dataset for High-Level Synthesis Functional Verification through LLM-Aided Bug Injection," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10691860/>
- [30] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Evaluating LLMs for Hardware Design and Test," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10691811/>
- [31] A. Sharma, T.-D. Ene, K. Kunal, M. Liu, Z. Hasan, and H. Ren, "Assessing Economic Viability: A Comparative Analysis of Total Cost of Ownership for Domain-Adapted Large Language Models versus State-of-the-art Counterparts in Chip Design Coding Assistance," in *2024 IEEE LLM Aided Design Workshop (LAD)*. San Jose, CA, USA: IEEE, Jun. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10691849/>
- [32] Acmeism. (2025) Rosettacodedata. [Online]. Available: <https://github.com/acmeism/RosettaCodeData>
- [33] M. Eslami, B. Ghavami, M. Raji, and A. Mahani, "A survey on fault injection methods of digital integrated circuits," *Integration*, vol. 71, pp. 154–163, 2020.
- [34] M. Leeke, "Simultaneous fault models for the generation and location of efficient error detection mechanisms," *The Computer Journal*, vol. 63, no. 5, pp. 758–773, 2020.
- [35] Z. Zou, T. Jiang, Y. Wang, T. Xue, N. Zhang, and J. Luan, "Code vulnerability detection based on augmented program dependency graph and optimized codebert," *Scientific Reports*, vol. 15, no. 1, p. 39301, 2025.